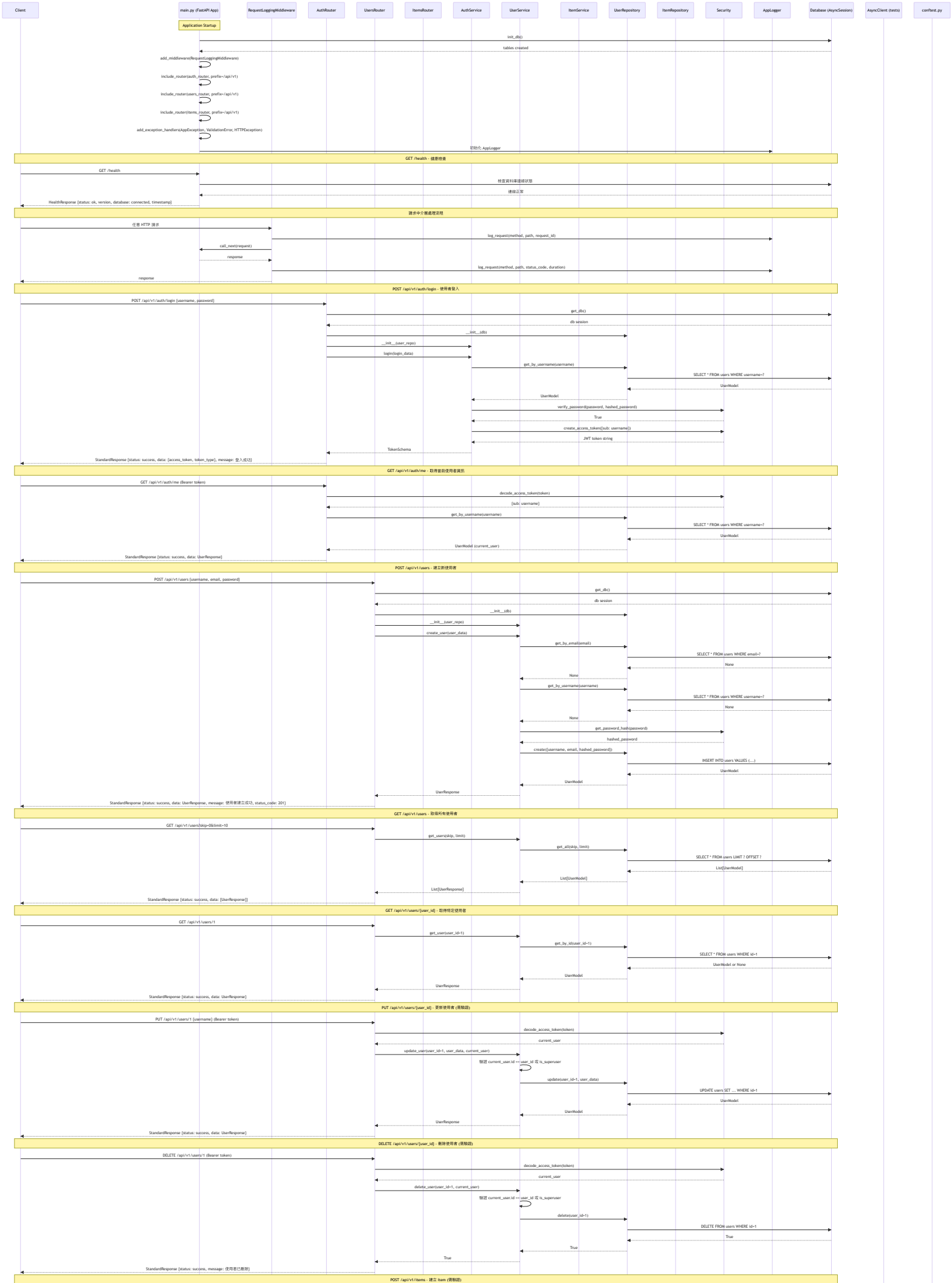
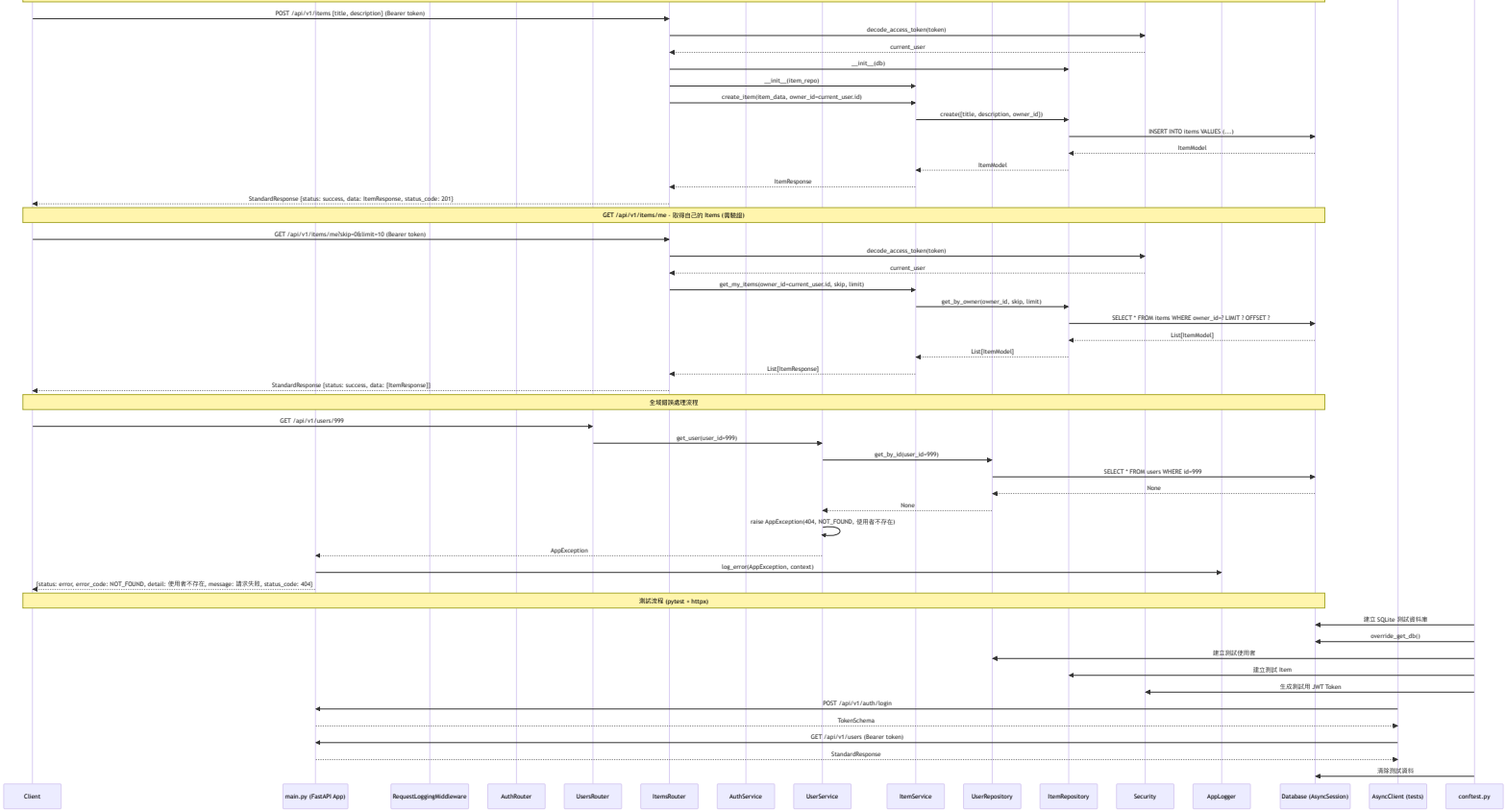

Client
main.py (FastAPI App)
RequestLoggingMiddleware
AuthRouter
UsersRouter
ItemsRouter
AuthService
UserService
ItemService
UserRepository
ItemRepository
Security
AppLogger
Database (AsyncSession)
AsyncClient (tests)
conftest.py
Application Startup
init_db()
tables created
add_middleware(RequestLoggingMiddleware)
include_router(auth_router, prefix=/api/v1)
include_router(users_router, prefix=/api/v1)
include_router(items_router, prefix=/api/v1)
add_exception_handlers(AppException, ValidationError, HTTPException)
初始化 AppLogger
GET /health - 健康檢查
GET /health
檢查資料庫連線狀態
連線正常
HealthResponse {status: ok, version, database: connected, timestamp}
請求中介層處理流程
任意 HTTP 請求
log_request(method, path, request_id)
call_next(request)
response
log_request(method, path, status_code, duration)
response
POST /api/v1/auth/login - 使用者登入
POST /api/v1/auth/login {username, password}
get_db()
db session
__init__(db)
__init__(user_repo)
login(login_data)
get_by_username(username)
SELECT * FROM users WHERE username=?
UserModel
UserModel
verify_password(password, hashed_password)
True
create_access_token({sub: username})
JWT token string
TokenSchema
StandardResponse {status: success, data: {access_token, token_type}, message: 登入成功}
GET /api/v1/auth/me - 取得當前使用者資訊
GET /api/v1/auth/me (Bearer token)
decode_access_token(token)
{sub: username}
get_by_username(username)
SELECT * FROM users WHERE username=?
UserModel
UserModel (current_user)
StandardResponse {status: success, data: UserResponse}
POST /api/v1/users - 建立新使用者
POST /api/v1/users {username, email, password}
get_db()
db session
__init__(db)
__init__(user_repo)
create_user(user_data)
get_by_email(email)
SELECT * FROM users WHERE email=?
None
None
get_by_username(username)
SELECT * FROM users WHERE username=?
None
None
get_password_hash(password)
hashed_password
create({username, email, hashed_password})
INSERT INTO users VALUES (...)
UserModel
UserModel
UserResponse
StandardResponse {status: success, data: UserResponse, message: 使用者建立成功, status_code: 201}
GET /api/v1/users - 取得所有使用者
GET /api/v1/users?skip=0&limit=10
get_users(skip, limit)
get_all(skip, limit)
SELECT * FROM users LIMIT ? OFFSET ?
List[UserModel]
List[UserModel]
List[UserResponse]
StandardResponse {status: success, data: [UserResponse]}
GET /api/v1/users/{user_id} - 取得特定使用者
GET /api/v1/users/1
get_user(user_id=1)
get_by_id(user_id=1)
SELECT * FROM users WHERE id=1
UserModel or None
UserModel
UserResponse
StandardResponse {status: success, data: UserResponse}
PUT /api/v1/users/{user_id} - 更新使用者 (需驗證)
PUT /api/v1/users/1 {username} (Bearer token)
decode_access_token(token)
current_user
update_user(user_id=1, user_data, current_user)
驗證 current_user.id == user_id 或 is_superuser
update(user_id=1, user_data)
UPDATE users SET ... WHERE id=1
UserModel
UserModel
UserResponse
StandardResponse {status: success, data: UserResponse}
DELETE /api/v1/users/{user_id} - 刪除使用者 (需驗證)
DELETE /api/v1/users/1 (Bearer token)
decode_access_token(token)
current_user
delete_user(user_id=1, current_user)
驗證 current_user.id == user_id 或 is_superuser
delete(user_id=1)
DELETE FROM users WHERE id=1
True
True
True
StandardResponse {status: success, message: 使用者已刪除}
POST /api/v1/items - 建立 Item (需驗證)


POST /api/v1/items {title, description} (Bearer token)
decode_access_token(token)
current_user
__init__(db)
__init__(item_repo)
create_item(item_data, owner_id=current_user.id)
create({title, description, owner_id})
INSERT INTO items VALUES (...)
ItemModel
ItemModel
ItemResponse
StandardResponse {status: success, data: ItemResponse, status_code: 201}
GET /api/v1/items/me - 取得自己的 items (需驗證)
GET /api/v1/items/me?skip=0&limit=10 (Bearer token)
decode_access_token(token)
current_user
get_my_items(owner_id=current_user.id, skip, limit)
get_by_owner(owner_id, skip, limit)
SELECT * FROM items WHERE owner_id=? LIMIT ? OFFSET ?
List[ItemModel]
List[ItemModel]
List[ItemResponse]
StandardResponse {status: success, data: [ItemResponse]}
全域錯誤處理流程
GET /api/v1/users/999
get_user(user_id=999)
get_by_id(user_id=999)
SELECT * FROM users WHERE id=999
None
None
raise AppException(404, NOT_FOUND, 使用者不存在)
AppException
log_error(AppException, context)
{status: error, error_code: NOT_FOUND, detail: 使用者不存在, message: 請求失敗, status_code: 404}
測試流程 (pytest + httpx)
建立 SQLite 測試資料庫
override_get_db()
建立測試使用者
建立測試 Item
生成測試用 JWT Token
POST /api/v1/auth/login
TokenSchema
GET /api/v1/users (Bearer token)
StandardResponse
清除測試資料
Client
main.py (FastAPI App)
RequestLoggingMiddleware
AuthRouter
UsersRouter
ItemsRouter
AuthService
UserService
ItemService
UserRepository
ItemRepository
Security
AppLogger
Database (AsyncSession)
AsyncClient (tests)
conftest.py